Cours 10
Programmation logique, Résolution, unification

Logique – Licence Informatique



Contexte du cours

Jusque maintenant...

- Logique propositionnelle
  - Syntaxe et sémantique (cours 1-3)
  - Preuve formelle : déduction naturelle (cours 4-5)
  - Automatisation et application : solveurs SAT (cours 6)
- Logique des prédicats du premier ordre
  - Syntaxe et sémantique (cours 7-8)
  - Preuve formelle : déduction naturelle (cours 9)
  - Automatisation et application ?

Contexte du cours

Jusque maintenant...

- Logique propositionnelle
  - Syntaxe et sémantique (cours 1-3)
  - Preuve formelle : déduction naturelle (cours 4-5)
  - Automatisation et application : solveurs SAT (cours 6)
- Logique des prédicats du premier ordre
  - Syntaxe et sémantique (cours 7-8)
  - Preuve formelle : déduction naturelle (cours 9)
  - Automatisation et application ?
    `prolog` : programmation logique (fondé sur des preuves par résolution)

prolog - Historique et influence

- Un langage né de la logique et de l'IA (années 1970)
  - Créé à Marseille (Alain Colmerauer, Philippe Roussel)
    Principe : *programmer, c'est énoncer des faits logiques et laisser la machine raisonner.*
  - Utilisé dans les premiers systèmes experts, NLP, planification.
- Langage influent
  - Nombreuses extensions (programmation par contraintes, extensions probabilistes, programmation logique abductive, apprentissage symbolique...)
  - Principes et syntaxe repris par outils (Theorem prover, Answer Set Programming...)
- Usages
  - Raisonnement et démonstration automatique (système expert, assistant de preuve, vérification de conformité)
  - Raisonnement par contraintes (planification, scheduling, optimisation combinatoire)
  - IA symbolique et hybride

La programmation logique

- Base des approches déclaratives
- Un nouveau paradigme de programmation
  - Impératif (on dit comment faire) – `python, C, java`
  - Fonctionnel (on décrit ce que c'est) – `Caml, lisp, Haskell`
  - Logique (on énonce ce qui est vrai) – `prolog`
- Originalité :
  - Pas d'instructions, pas de flux de contrôle explicite.
  - L'exécution est une déduction logique automatique.
- Fondements logiques :
  - Basé sur la logique des prédicats et les clauses de Horn.
  - Le moteur Prolog implémente la **résolution** SLD (nécessite mécanisme d'**unification**).

La programmation logique

Principe

- On décrit ce qui est connu comme vrai (programme logique)
  - Déclarer des faits :

    ```
    parent(paul, jean).
    parent(jean, pierre).
    ```
  - Définir des règles :

    ```
    grandparent(X,Y) :- parent(X,Z), parent(Z,Y).
    ```
- On pose une question (requête).

  ```
  ?- grandparent(paul,X).
  ```
- Le moteur cherche une preuve logique (résolution par unification et backtracking).
  - Réponse : instantiation des variables rendant la requête prouvable.

    ```
    X = pierre
    ```

Plan

1. Programmation logique
   - Présentation
   - Exemple
   - Syntaxe
   - Moteur de prolog
2. La règle de résolution
   - En logique propositionnelle
   - En LPPO
   - En prolog
3. Unification en LPPO
4. Raisonnement par résolution
   - Preuves par résolution
   - Stratégie de prolog

Prolog : Présentation

Dans l'UE, on utilise `SWI-prolog`.
Téléchargement : `https://www.swi-prolog.org/`

Programmes et requêtes

- Définir ce qui est vrai : **Programme**
  le 'code' à proprement parler (fichiers `.pl`)
- Poser des questions : **Requêtes**
  L'interface de prolog (mode query)

Prolog : Présentation

- Programme – Mode *consult*.
  - Contient les faits et les règles qui définissent le problème.
  - Il est demandé de mettre ensemble les règles ayant le même prédicat en tête (facilite le débogage).
  - Edité à part et enregistré dans un fichier "`.pl`".
- Requêtes – Mode *query*.
  - C'est l'invite de commande quand on lance prolog
  - Une requête est un prédicat, ou un séquence de prédicats (interprétés conjonctivement).
  - Si la requête contient des variables, prolog essaie de trouver une substitution permettant de la résoudre (si nécessaire).
  - Les commandes natives de prolog sont exprimées comme des prédicats :
    - ★ `[`*filename*`].` permet de charger un programme.
    - ★ `halt.` permet de quitter prolog.

Exemple

Programme `famille.pl`

```
femme(alice).
femme(victoria).
homme(albert).
homme(edward).
parents(edward, victoria, albert).
parents(alice, victoria, albert).
soeur(X,Y) :- femme(X),
              parents(X,Mere,Pere),
              parents(Y,Mere,Pere).
```

Mode query

```
?-[famille.pl].
true.

?-femme(alice).
```

Exemple

Programme `famille.pl`

```
femme(alice).
femme(victoria).
homme(albert).
homme(edward).
parents(edward, victoria, albert).
parents(alice, victoria, albert).
soeur(X,Y) :- femme(X),
              parents(X,Mere,Pere),
              parents(Y,Mere,Pere).
```

Mode query

```
?-[famille.pl].
true.

?-femme(alice).
true.

?-femme(albert).
```

Exemple

Programme `famille.pl`

```
femme(alice).
femme(victoria).
homme(albert).
homme(edward).
parents(edward, victoria, albert).
parents(alice, victoria, albert).
soeur(X,Y) :- femme(X),
              parents(X,Mere,Pere),
              parents(Y,Mere,Pere).
```

Mode query

```
?-[famille.pl].
true.

?-femme(alice).
true.

?-femme(albert).
false.

?-homme(X).
```

Exemple

Programme `famille.pl`

```
femme(alice).
femme(victoria).
homme(albert).
homme(edward).
parents(edward, victoria, albert).
parents(alice, victoria, albert).
soeur(X,Y) :- femme(X),
              parents(X,Mere,Pere),
              parents(Y,Mere,Pere).
```

Mode query

```
?-[famille.pl].
true.

?-femme(alice).
true.

?-femme(albert).
false.

?-homme(X).
X=albert
```

Exemple

Programme `famille.pl`

```
femme(alice).
femme(victoria).
homme(albert).
homme(edward).
parents(edward, victoria, albert).
parents(alice, victoria, albert).
soeur(X,Y) :- femme(X),
              parents(X,Mere,Pere),
              parents(Y,Mere,Pere).
```

Mode query

```
?-[famille.pl].
true.

?-femme(alice).
true.

?-femme(albert).
false.

?-homme(X).
X=albert ;
X=edward.

?-femme(X), parents(X,U,I).
```

Exemple

Programme `famille.pl`

```
femme(alice).
femme(victoria).
homme(albert).
homme(edward).
parents(edward, victoria, albert).
parents(alice, victoria, albert).
soeur(X,Y) :- femme(X),
              parents(X,Mere,Pere),
              parents(Y,Mere,Pere).
```

Mode query

```
?-[famille.pl].
true.

?-femme(alice).
true.

?-femme(albert).
false.

?-homme(X).
X=albert ;
X=edward.

?-femme(X), parents(X,U,I).
X=alice, U=victoria, I=albert
```

Exemple

Programme `famille.pl`

```
femme(alice).
femme(victoria).
homme(albert).
homme(edward).
parents(edward, victoria, albert).
parents(alice, victoria, albert).
soeur(X,Y) :- femme(X),
              parents(X,Mere,Pere),
              parents(Y,Mere,Pere).
```

Mode query

```
?-[famille.pl].
true.

?-femme(alice).
true.

?-femme(albert).
false.

?-homme(X).
X=albert ;
X=edward.

?-femme(X), parents(X,U,I).
X=alice, U=victoria, I=albert ;
false.

?-soeur(U, V).
```

Exemple

Programme `famille.pl`

```
femme(alice).
femme(victoria).
homme(albert).
homme(edward).
parents(edward, victoria, albert).
parents(alice, victoria, albert).
soeur(X,Y) :- femme(X),
              parents(X,Mere,Pere),
              parents(Y,Mere,Pere).
```

Mode query

```
?-[famille.pl].
true.

?-femme(alice).
true.

?-femme(albert).
false.

?-homme(X).
X=albert ;
X=edward.

?-femme(X), parents(X,U,I).
X=alice, U=victoria, I=albert ;
false.

?-soeur(U, V).
U=alice, V=edward
```

Exemple

Programme `famille.pl`

```
femme(alice).
femme(victoria).
homme(albert).
homme(edward).
parents(edward, victoria, albert).
parents(alice, victoria, albert).
soeur(X,Y) :- femme(X),
              parents(X,Mere,Pere),
              parents(Y,Mere,Pere).
```

Mode query

```
?-[famille.pl].
true.

?-femme(alice).
true.

?-femme(albert).
false.

?-homme(X).
X=albert ;
X=edward.

?-femme(X), parents(X,U,I).
X=alice, U=victoria, I=albert ;
false.

?-soeur(U, V).
U=alice, V=edward ;
U=alice, V=alice
```

Exemple

Programme `famille.pl`

```
femme(alice).
femme(victoria).
homme(albert).
homme(edward).
parents(edward, victoria, albert).
parents(alice, victoria, albert).
soeur(X,Y) :- femme(X),
              parents(X,Mere,Pere),
              parents(Y,Mere,Pere).
```

Mode query

```
?-[famille.pl].
true.

?-femme(alice).
true.

?-femme(albert).
false.

?-homme(X).
X=albert ;
X=edward.

?-femme(X), parents(X,U,I).
X=alice, U=victoria, I=albert ;
false.

?-soeur(U, V).
U=alice, V=edward ;
U=alice, V=alice ;
false.
```

Non-directionalité des définitions

Il peut y avoir plusieurs réponses à une requête :

- Si '.' à la fin : `prolog` sait qu'il n'y a pas d'autres réponses.
- Sinon, on peut taper ' ;' pour en afficher d'autres
  - '`false.`' à la fin signifie qu'il n'y a plus d'autres réponses (mais que `prolog` ne le savait pas avant de les chercher)

Une définition peut répondre à plusieurs questions :

```
?-soeur(X, edward).
```

Qui sont les soeurs d'Edward ?

```
X=alice ; false.
```

```
?-soeur(alice, X).
```

Qui a Alice pour soeur ?

```
X=edward ; X= alice ; false.
```

```
?-addition(s(0), s(s(0)),X).
```

Combien font 1+2 ?

```
X=s(s(s(0))).
```

```
?-addition(X, Y, s(s(0))).
```

Quels sont les (x,y) tels que x+y=2 ?

```
X=0,Y=s(s(0)); X=s(0),Y=s(0);X=s(s(0)),Y=0.
```

Prolog : Syntaxe

- Termes (apparaissent comme arguments d'un prédicat ou d'un opérateur prolog infixe – par ex. `=`, `==`, `is`...)
  - Constantes : chaînes de caractères commençant par une minuscule (non suivie de parenthèses)
    Exemples : `alice`, `edward`, `james_bond_007`, `a`, `x`, `x12`
  - Nombres : interprétés comme constantes, mais prolog contient des primitives arithmétiques
    Exemples : `0`, `12`, `-1`...
  - Variables : chaînes de caractères commençant par une majuscule ou par _
    Exemples : `X`, `Mere`, `_G341`, `Stranger`, `The_Older`, `A`, `X12`
  - Fonctions : chaînes de caractères commençant par une minuscule (suivie de parenthèses avec les arguments)
    Exemples : `f(a,b)`, `s(s(0))`, `pireEnnemi(X)`, ...
  - Listes : expression encadrée par `[ ]` (voir plus tard)

Prolog : Syntaxe

- Prédicats : commencent aussi par minuscules (la différence avec les fonctions se fait par la position : les prédicats sont dans une règle, les fonctions dans un prédicat)
  Exemples : `parents(X,Y,Z)`, `addition(X,Y,s(s(0)))`
  (`addition` : prédicat, `s` : fonction) ...
- Règles : chaque ligne du programme. Commence par un prédicat et finit par un point.
  - Faits : juste un prédicat suivi d'un point.
    Exemples : `parents(edward, victoria, albert)`, `addition(s(0),s(0),s(s(0))).`, `addition(0,X,X).`
  - Règle : tête de la règle (conclusion), suivi de `:-` (qui représente $\leftarrow$ – implication de la droite vers la gauche), d'un corps de règle et d'un point.
    Exemples : `soeur(X,Y) :- femme(X), parents(X,Mere,Pere), parents(Y,Mere,Pere).`,

Unification vs égalité

- $exp_1 = exp_2$ vérifié si exp_1 est unifiable avec exp_2

```
?-a=a. true. / ?- X=a. X=a. / ?-X=Y. X=Y.
?-f(X,Y)=f(g(Y,b),a). X=g(a,b),Y=a.
?-f(X,X)=f(g(Y,b),a). false.
```

- $exp_1 == exp_2$ vérifié si exp_1 est égal à exp_2

```
?-a==a. true. / ?- X==a. false. / ?-X==Y. false.
?-f(X,Y)==f(X,Y). true. / ?-f(X,Y)==f(X,X). false.
```

- Si la variable est déjà instantiée, l'égalité se base sur sa valeur actuelle.

```
?- X=a, X==a. X=a.
?- homme(X), X==albert. X=albert ; false.
```

Unification vs égalité

Note : il est souvent préférable et plus concis de juste réutiliser le même nom de variable plutôt que d'utiliser '=' ou '=='.
Exemple :

```
soeur(X,Y) :- femme(X), parents(X,P1,M1),
parents(Y,P2,M2),P1==P2, M1==M2.
```

Il est préférable de ne pas introduire `P2,M2`, mais plutôt de réutiliser les variables dont la valeur est souhaitée.

```
soeur(X,Y) :- femme(X), parents(X,P1,M1),
parents(Y,P1,M1).
```

Différent de

on peut utiliser `\==` pour tester si deux valeurs sont différentes.

```
soeur(X,Y) :- femme(X), parents(X,P1,M1),
parents(Y,P1,M1),X\==Y.
```

Listes

Constructeurs de listes :

- Liste vide : `[]`
- Constructeur de liste : `[Tete|Queue]` où `Tete` doit être un élément de la liste et Queue une liste.

  `?-[T|Q]=[a,b,c,d].` `T=a, Q=[b,c,d].`

  `?-[T|Q]=[a,b].` `T=a,Q=[b].` / `?-[T|Q]=[].` `false.`

  `?-[T|Q]=[a]` `T=a,Q=[]` / `?-[T|Q]=a.` `false.`
- Liste à plusieurs éléments : `[X]` (liste à 1 élément), `[X,Y]` (liste à deux éléments)

  `?-[X,Y]=[a,b,c,d].` `false.` / `?-` `[X,Y]=[a,b].` `X=a,Y=b.`

  `?-[X,Y,Z]=[[a,b,c],[2,3],[]].` `X=[a,b,c],Y=[2,3],Z=[].`
- Récupérer les n premier éléments : `[X1,X2,...,Xn|Reste]` (liste commençant par `X1`, ... `Xn` et se poursuivant par les éléments de la liste `Reste`.

  `?-[X,Y|Q]=[a].` `false.` / `?-` `[X,Y|Q]=[a,b].` `X=a,Y=b,Q=[].`

  `?-[X,Y|Q]=[a,b,c,d].` `X=a,Y=b,Q=[c,d].`

Prolog : Moteur

Pour répondre à une requête, prolog cherche une preuve par résolution :

1. Cherche une règle ou un fait du programme dont la tête s'**unifie** avec la requête (en utilisant la stratégie dite **SLD**).
2. S'il en trouve : **résout** la règle avec la requête, pour obtenir une nouvelle requête en remplaçant la requête par le résultat de la résolution. Itère jusqu'à ce que la requête soit vide (solution).
3. S'il n'y en a pas : backtrack pour utiliser une autre règle dans une résolution précédente. Répond faux si impossible.

Nous allons donc maintenant expliciter les éléments de ce moteur

- La résolution (en proposition, LPPO et prolog)
- Algorithme d'unification
- Preuves par résolution et stratégies pour automatisation.

Résolution : une généralisation du Modus Ponens

Règle d'inférence
Modus ponens classique

$$\frac{A \to B \qquad A}{B}$$

Résolution : une généralisation du Modus Ponens

Règle d'inférence
Si on n'est pas sûr de la prémisse

$$\frac{A \rightarrow B \qquad A \vee C}{???}$$

Résolution : une généralisation du Modus Ponens

Règle d'inférence

Réécriture $D = \neg C$

$$\frac{A \to B \qquad D \to A}{D \to B}$$

Résolution : une généralisation du Modus Ponens

Règle d'inférence
Sous forme disjonctive

$$\frac{\neg A \vee B \qquad \neg D \vee A}{\neg D \vee B}$$

Résolution : une généralisation du Modus Ponens

Règle d'inférence

Sous forme disjonctive (avec $C = \neg D$)

$$\frac{\neg A \vee B \qquad A \vee C}{B \vee C}$$

Résolution : une généralisation du Modus Ponens

Règle d'inférence

Avec $B = L_1 \vee \ldots \vee L_n$ et $C = L'_1 \vee \ldots \vee L'_k$ où chaque L_i ou L'_i est un littéral (p ou $\neg p$)

$$\frac{\neg A \vee L_1 \vee \ldots \vee L_n \qquad A \vee L'_1 \vee \ldots \vee L'_k}{L_1 \vee \ldots \vee L_n \vee L'_1 \vee \ldots \vee L'_k}$$

Résolution : une généralisation du Modus Ponens

Règle d'inférence

$$\frac{C_1 : \neg A \vee L_1 \vee \ldots \vee L_n \qquad C_2 : A \vee L'_1 \vee \ldots \vee L'_k}{R : L_1 \vee \ldots \vee L_n \vee L'_1 \vee \ldots \vee L'_k}$$

C'est la règle de résolution en logique propositionnelle.

- C_1, C_2 sont appelées les prémisses de cette résolution
- A est le littéral sur lequel on fait la résolution
- R est appelé la résolvante de cette résolution. On note $R = Res(C_1, C_2, A)$

Résolution : une généralisation du Modus Ponens

Règle d'inférence

Simplification de R si $L_1 = L'_1 = C_3$

$$\frac{C_1 : \neg A \vee C_3 \vee \ldots \vee L_n \qquad C_2 : A \vee C_3 \vee \ldots \vee L'_k}{R : C_3 \vee \ldots \vee L_n \vee \ldots \vee L'_k}$$

C'est la règle de résolution en logique propositionnelle.

- C_1, C_2 sont appelées les prémisses de cette résolution
- A est le littéral sur lequel on fait la résolution
- R est appelé la résolvante de cette résolution. On note $R = Res(C_1, C_2, A)$

Résolution en logique propositionnelle

La règle de résolution

Soit p une variable propositionnelle et $C_1 : l_1 \vee \ldots l_n$, $C_2 : l'_1 \vee \ldots l'_k$ deux clauses (disjonctions de littéraux). Résolution de $p \vee C_1$ et $\neg p \vee C_2$ sur p :

$$\frac{p \vee C_1 \qquad \neg p \vee C_2}{C_1 \vee C_2}$$

écriture linéaire : $Res(p \vee C_1, \neg p \vee C_2, p) = C_1 \vee C_2$

Exemples

$a \vee b$ et $\neg a \vee b$ $\qquad$ $\neg a \vee b$ et $a \vee \neg b$ $\qquad$ $\neg a \vee b \vee c$ et $\neg b$

Résolution en logique propositionnelle

La règle de résolution

Soit p une variable propositionnelle et $C_1 : l_1 \vee \ldots l_n$, $C_2 : l'_1 \vee \ldots l'_k$ deux clauses (disjonctions de littéraux). Résolution de $p \vee C_1$ et $\neg p \vee C_2$ sur p :

$$\frac{p \vee C_1 \qquad \neg p \vee C_2}{C_1 \vee C_2}$$

écriture linéaire : $Res(p \vee C_1, \neg p \vee C_2, p) = C_1 \vee C_2$

Exemples

$a \vee b$ et $\neg a \vee b$
$b \vee b = b$

$\neg a \vee b$ et $a \vee \neg b$
sur a :$b \vee \neg b = \top$

$\neg a \vee b \vee c$ et $\neg b$
$\neg a \vee c$

Cas de la LPPO

Syllogisme

Tout les hommes sont mortels

Socrate est un homme

$\Rightarrow$ Socrate est mortel

Cas de la LPPO

Syllogisme

Tout les hommes sont mortels
$R : \forall X(H(X) \rightarrow M(X))$
Socrate est un homme
$F : H(s)$

$\Rightarrow$ Socrate est mortel
$C : M(s)$

Note : *Dans ce cours, par homogénéité avec la syntaxe* `prolog` *le choix est fait de faire commencer les symboles de variables par des majuscules, contrairement à l'usage des cours précédents.*

Cas de la LPPO

Syllogisme

Tout les hommes sont mortels
$R : \forall X(H(X) \rightarrow M(X))$
Socrate est un homme
$F : H(s)$

$\Rightarrow$ Socrate est mortel
$C : M(s)$

Note : *Dans ce cours, par homogénéité avec la syntaxe* `prolog` *le choix est fait de faire commencer les symboles de variables par des majuscules, contrairement à l'usage des cours précédents.*

Preuve LPPO

On mobilise deux règles d'inférences :

- Instantiation de R avec $\sigma = \{X \mapsto s\}$ donne $\sigma(R) = H(s) \rightarrow M(s)$
- Modus ponens simple entre $\sigma(R)$ et F donne $C : M(s)$

Résolution en LPPO

Littéral : une formule atomique ($P(t_1, \ldots, t_k)$ où P prédicat d'arité k) ou sa négation
Clauses : disjonction universellement quantifiées de littéraux (peut s'écrire comme un ensemble de littéraux)

Ici : $C_R : \neg H(X) \vee M(X)$ et $C_F : H(s)$.

On combine les deux étapes : trouver σ tel que $\sigma(C_R)$ et $\sigma(C_F)$ ait deux littéraux complémentaires, puis appliquer la résolution comme en propositionnel sur $\sigma(C_R)$ et $\sigma(C_F)$.

$$\frac{\neg H(X) \vee M(X) \qquad H(s)}{M(s)} \{X \mapsto s\}$$

Résolution en LPPO

La règle de résolution

Soit $L_{pos} = P(t_1, \ldots, t_k)$ et $L_{neg} = \neg P(t'_1, \ldots, t'_k)$ deux littéraux sur le même prédicats P d'arité k tels que L_{pos} et $\neg L_{neg}$ $(P(t'_1, \ldots, t'_k))$ sont unifiables ($\exists \sigma$ telle que $\sigma(L_{pos}) \equiv \sigma(\neg L_{neg})$).
Résolution de $L_1 \vee C_1$ et $L_2 \vee C_2$ sur σ :

$$\frac{L_{pos} \vee C_1 \qquad L_{neg} \vee C_2}{\sigma(C_1) \vee \sigma(C_2)}\ \sigma$$

écriture linéaire : $Res(L_{pos} \vee C_1, L_{neg} \vee C_2, \sigma) = \sigma(C_1) \vee \sigma(C_2)$

Exemples

$$(1) : \neg P(X) \vee R(X) \text{ et } P(f(a)) \vee P(b)$$

$$(2) : \neg P(f(X), f(a)) \vee R(X, b) \text{ et } P(f(f(X)), Y) \vee R(f(a), Y)$$

Résolution en LPPO

La règle de résolution

Soit $L_{pos} = P(t_1, \ldots, t_k)$ et $L_{neg} = \neg P(t'_1, \ldots, t'_k)$ deux littéraux sur le même prédicats P d'arité k tels que L_{pos} et $\neg L_{neg}$ $(P(t'_1, \ldots, t'_k))$ sont unifiables ($\exists \sigma$ telle que $\sigma(L_{pos}) \equiv \sigma(\neg L_{neg})$).
Résolution de $L_1 \vee C_1$ et $L_2 \vee C_2$ sur σ :

$$\frac{L_{pos} \vee C_1 \qquad L_{neg} \vee C_2}{\sigma(C_1) \vee \sigma(C_2)}\ \sigma$$

écriture linéaire : $Res(L_{pos} \vee C_1, L_{neg} \vee C_2, \sigma) = \sigma(C_1) \vee \sigma(C_2)$

Exemples

$$(1) : \neg P(X) \vee R(X) \text{ et } P(f(a)) \vee P(b)$$
$$\text{Res} : R(f(a)) \vee P(b)\ (X \mapsto f(a)) \text{ et } R(b) \vee P(f(a))\ (X \mapsto b)$$
$$(2) : \neg P(f(X), f(a)) \vee R(X, b) \text{ et } P(f(f(X)), Y) \vee R(f(a), Y)$$

Résolution en LPPO

La règle de résolution

Soit $L_{pos} = P(t_1, \ldots, t_k)$ et $L_{neg} = \neg P(t'_1, \ldots, t'_k)$ deux littéraux sur le même prédicats P d'arité k tels que L_{pos} et $\neg L_{neg}$ $(P(t'_1, \ldots, t'_k))$ sont unifiables ($\exists \sigma$ telle que $\sigma(L_{pos}) \equiv \sigma(\neg L_{neg})$).
Résolution de $L_1 \vee C_1$ et $L_2 \vee C_2$ sur σ :

$$\frac{L_{pos} \vee C_1 \qquad L_{neg} \vee C_2}{\sigma(C_1) \vee \sigma(C_2)}\ \sigma$$

écriture linéaire : $Res(L_{pos} \vee C_1, L_{neg} \vee C_2, \sigma) = \sigma(C_1) \vee \sigma(C_2)$

Exemples

$$(1) : \neg P(X) \vee R(X) \text{ et } P(f(a)) \vee P(b)$$
$$\text{Res} : R(f(a)) \vee P(b)\ (X \mapsto f(a)) \text{ et } R(b) \vee P(f(a))\ (X \mapsto b)$$
$$(2) : \neg P(f(X), f(a)) \vee R(X, b) \text{ et } P(f(f(X_2)), Y) \vee R(f(a), Y)$$
$$\text{Res} : R(f(X_2), b) \vee R(f(a), f(a))\ (X \mapsto f(X_2), Y \mapsto f(a))$$

Cas des programmes logiques

Soit une clause $C_1 = P_0 \vee \ldots P_{k-1} \vee \neg P_k \vee \ldots \neg P_n$. Elle peut se réécrire en une règle logiquement équivalente
$R_1 = (P_k \wedge \ldots \wedge P_n) \rightarrow (P_0 \vee \ldots \vee P_{k-1})$

On appelle **programme logique** un ensemble de règles
Par convention, on met la règle en tête, on utilise `:-` pour représenter $\leftarrow$, les `;` en tête correspondent à des $\vee$ et les `,` dans le corps à des $\wedge$:

`P`$_0$`;...;P`$_{k-1}$ `:-` `P`$_k$`,...,P`$_n$`.`

Programme `prolog`
On impose $k = 1$ (un seul litéral positif dans la clause) :

`P`$_0$ `:-` `P`$_1$`,...,P`$_n$`.`

Cas des programmes logiques

Programme logique en `prolog`

Un programme `prolog` peut s'interpréter comme un ensemble de clauses :

Fait	$\texttt{A}_0\texttt{.}$	$\mapsto$	A_0
Règle	$\texttt{A}_0\ \texttt{:-}\ \texttt{B}_1\texttt{,}\ldots\texttt{,}\texttt{B}_k\texttt{.}$	$\mapsto$	$A_0 \vee \neg B_1 \ldots \vee \neg B_k$

Une **requête** $\mathcal{Q}$ de la forme $\texttt{Q}_1\texttt{,}\ldots\texttt{,}\texttt{Q}_s$ est une conjonction de formules atomiques ($\mathcal{Q} = Q_1 \wedge \ldots \wedge Q_s$).

On procède par réfutation : ajout de $\neg\mathcal{Q}$ (clause) au programme P pour prouver la clause vide $P \cup \{\neg\mathcal{Q}\} \vdash \square$.

On utilise `?-` pour noter cette réfutation.

Requête	$\texttt{?-}\ \texttt{Q}_1\texttt{,}\ldots\texttt{,}\texttt{Q}_s\texttt{.}$	$\mapsto$	$\neg Q_1 \ldots \vee \neg Q_s$

`prolog` cherche à construire des preuves en résolvant une requête avec une règle (ou un fait)

Résolution en `prolog`

On veut résoudre la requête `?-` $\texttt{Q}_1\texttt{,}\ldots\texttt{,}\texttt{Q}_s\texttt{.}$ avec une règle
$\texttt{Q}_1$ `:-` $\texttt{P}_1\texttt{,}\ldots\texttt{,}\texttt{P}_k\texttt{.}$
Version clausale :

$$\frac{\neg Q_1 \vee \ldots \neg Q_s \qquad Q_1 \vee \neg P_1 \vee \ldots \vee \neg P_k}{\neg P_1 \vee \ldots \vee \neg P_k \vee \neg Q_2 \vee \ldots \neg Q_s} \{X \mapsto \ldots\}$$

Résolution en `prolog`

On veut résoudre la requête `?- Q1,...,Qs.` avec une règle `Q1 :- P1,...,Pk.`
Avec les notations prolog :

$$\frac{\texttt{?- } Q_1\texttt{, ..., } Q_s \qquad Q_1\texttt{ :- } P_1\texttt{, ..., } P_k}{\texttt{?- } P_1\texttt{, ..., } P_k\texttt{, } Q_2\texttt{, ..., } Q_s}\ \texttt{X=...}$$

Résolution en `prolog`

On veut résoudre la requête `?-` Q_1`,...,`Q_s`.` avec une règle Q_1`.`
Avec un fait (pas de corps)

$$\frac{\texttt{?-}\ Q_1\texttt{,}\ \ldots\texttt{,}\ Q_s \qquad Q_1}{\texttt{?-}\ Q_2\texttt{,}\ \ldots\texttt{,}\ Q_s}\ \texttt{X=...}$$

Résolution en `prolog`

On veut résoudre la requête `?-`Q_s`.` avec une règle Q_s`.`
Fin de preuve :

$$\frac{\texttt{?-}\ Q_s \qquad Q_s}{\square\ \texttt{(success)}}\ \texttt{X=...}$$

Unification

Définition

Deux formules atomiques F_1 et F_2 sont *unifiables* ssi il existe une substitution σ telle que $\sigma(F_1) = \sigma(F_2)$.

Pour effectuer une résolution en présence de variables (LPPO ou prolog), il faut unifier deux littéraux complémentaires et déterminer le σ correspondant.

Comment trouver (automatiquement) un tel σ s'il existe ?

Exemples

Exemple 1 : $P(f(X), g(Y, Y))$ et $P(Z, g(b, b))$
Exemple 2 : $P(f(X), Y)$ et $P(a, g(b, b))$
Exemple 3 : $P(f(X), g(Y, Y))$ et $P(a, g(a, b))$
Exemple 4 : $P(f(X), Y)$ et $P(X, g(b, b))$

Algorithme de [Martelli et al 1982]

Unification de $P(t_1, \ldots, t_k)$ et $P(t'_1, \ldots, t'_k)$

1. *Initialisation.* $G \leftarrow \{t_1 \doteq t'_1, \ldots, t_k \doteq t'_k\}$ et $\sigma = \emptyset$
2. Tant que G non vide, choisir $C = (\alpha \doteq \beta)$ dans G et :
   1. (swap) Si α est une fonction [a] et β une variable, les inverser
   2. (del) Si α et β sont égaux, retirer C de G
   3. Si $\alpha = f(t_1, \ldots, t_i)$ est une fonction
      - (conflict) Si $\beta = g(s_1, \ldots, s_j)$ est une fonction différente ($g \neq f$ ou $i \neq j$), renvoyer $\perp$ (Echec)
      - (decomp) Sinon ($\beta = f(s_1, \ldots, s_i)$), retirer C de G et y ajouter $\{t_1 \doteq s_1, \ldots, t_i \doteq s_i\}$
   4. Si $\alpha = X$ est une variable :
      - (check) Si X apparaît dans β : renvoyer $\perp$ (Echec)
      - (eliminate) Sinon, retirer C de G, remplacer X par β dans toutes les expressions de G et σ, puis ajouter $\{X \mapsto \beta\}$ à σ.
3. Renvoyer σ

a. Les constantes sont des fonctions d'arité 0

Exemple complexe

$\phi_1 : Enc(X, e(m(Y, e(Z))), m(e(X), e(a)))$ et
$\phi_2 : Enc(m(e(b), a), e(m(U, e(Z))), m(e(m(V, W)), Z))$
(soit $X \leq e^{Y.e^Z} \leq e^X.e^a$ et $e^b.a \leq e^{U.e^Z} \leq e^{V.W}.Z$)
Application de l'algo :(voir tableau)

Unification pour la résolution

Renommage

Les variables d'une clause étant universellement quantifiées, on peut renommer les variables avant de chercher à unifier des littéraux de 2 clauses complémentaires.

Exemple

$P(f(X), Y)$ et $P(X, b)$ ne sont pas unifiables .
Réso. $C_1 = P(f(X), Y) \vee Q(X, Y)$ et $C_2 = \neg P(X, b) \vee R(X)$?
On renomme X en Z dans C_2 et $\sigma = \{Z \mapsto f(X), Y \mapsto b\}$

$$\frac{P(f(X), Y) \vee Q(X, Y) \qquad \neg P(Z, b) \vee R(Z)}{Q(X, b) \vee R(f(X))}\ \sigma$$

Preuve par résolution

Définition

Etant donné un ensemble de clauses Σ (clauses d'entrées) et une clause C (la conclusion), une *preuve par résolution de C à partir de Σ* est un ensemble de clauses $R_1, \ldots, R_n$ tel que

- Pour chaque i, $R_i = Res(G_i, D_i, \sigma_i)$ où G_i et D_i sont pris dans $\Sigma \cup \{R_1, \ldots, R_{i-1}\}$
- $R_n = C$

On a alors

$$\Sigma \vdash_{res} C$$

Cette procédure est correcte et complète ($\Sigma \models C \Leftrightarrow \Sigma \vdash_{res} C$)

Exemples

$\Sigma_1 = \{p \vee q, \neg p \vee q, p \vee \neg q, \neg p \vee \neg q\}$
Prouver $\Sigma_1 \vdash_{res} \square$

$\Sigma_2 = \{\neg P(f(X)) \vee Q(X, Y), \neg Q(X, X) \vee P(X), P(f(f(a)))\}$
Prouver $\Sigma_2 \vdash_{res} P(a)$

Stratégies

Pour construire une preuve, il faut choisir à chaque étape deux clauses G_i et D_i.
$G_1 \in \Sigma$ est appelée la top-clause.
Preuve linéaire : $\forall i > 1$, $G_i = R_{i-1}$.

Preuve par réfutation
Soit une requête conjonctive $Q = \bigwedge_{i \in \{1,\ldots k\}} L_i$.
Pour prouver $\Sigma \models Q$, on prouve $\Sigma \cup \{\neg Q\} \vdash_{res} \square$
Typiquement, $\neg Q = \neg L_1 \vee \neg L_k$ est utilisée comme top-clause.

Stratégie de prolog

Pour répondre à requête `?-`Q_1`,...,`Q_s

1. Choix du littéral sur lequel faire la résolution : Q_1 (le premier).
2. Recherche d'une règle ou d'un fait dont la tête est unifiable à Q_1 avec substitution σ
   - s'il y en a, on choisit la première (non encore utilisée à cette étape) dans l'ordre du programme
     i. On résout la requête avec cette règle ou fait : on applique la substitution σ et on remplace la requête par sa résolvante
     ii. Si la nouvelle requête est vide : succès, on **renvoie** les substitutions appliquées aux variables de la requête initiale (ou `true` si pas de telles variables).
     iii. Sinon, on reprend (en 1) avec la nouvelle requête.
   - s'il n'y en pas (ou plus) : échec (local).
     i. Backtrack (si possible) : on annule la dernière résolution pour chercher une autre clause avec laquelle résoudre (2) (retour jusqu'à la plus récente décision pour laquelle un autre choix est possible)
     ii. Si pas de backtrack possible (plus de résolution annulable) : échec global, on **renvoie** `false`.

Stratégie de prolog

Cette stratégie est appelé **SLD-résolution** :

- Top-clause : $G_1 = \neg Q$ (la requête)
- Linéaire : Pour chaque $i > 1$, $G_i = R_{i-1}$ (on remplace la requête par la requête issue de la résolution choisie)
- Selective : Pour chaque i, on choisit (de façon déterministe) un littéral de G_i (le premier littéral de la requête) et on prend pour D_i une clause de Σ contenant un littéral complémentaire (règle ayant ce littéral en tête).
- Définie : car appliquée à un programme défini, c'est-à-dire un ensemble de clauses ayant au plus un littéral positif (ce qui est le cas d'un programme prolog avec une requête).

Propriété
La SLD-résolution est correcte est commplète pour les programmes définis.

Exemple

Programme :

```
1. p(X,Y) :- q(X,Z),r(Z,Y).
2. p(X,X) :- s(X).
3. q(b,a).
4. q(X,b).
5. q(X,a) :- r(a,X).
6. r(b,a).
7.  s(X) :- t(X,a).
8.  s(X) :- t(X,b).
9.  s(X) :- t(X,X).
10. t(a,b).
11. t(b,a).
```

Requête :

```
?-p(X,X).
```

Preuve :

Exemple

Programme :

```
1. p(X,Y) :- q(X,Z),r(Z,Y).
2. p(X,X) :- s(X).
3. q(b,a).
4. q(X,b).
5. q(X,a) :- r(a,X).
6. r(b,a).
7.  s(X) :- t(X,a).
8.  s(X) :- t(X,b).
9.  s(X) :- t(X,X).
10. t(a,b).
11. t(b,a).
```

Requête :

```
?-p(X,X).
```

Preuve :

```
?-q(X,Z),r(Z,X).    [1 - Y=X]
?-r(a,b).           [3 - X=b, Z=a]
fail.
```

Exemple

Programme :

```
1. p(X,Y) :- q(X,Z),r(Z,Y).
2. p(X,X) :- s(X).
3. q(b,a).
4. q(X,b).
5. q(X,a) :- r(a,X).
6. r(b,a).
7.  s(X) :- t(X,a).
8.  s(X) :- t(X,b).
9.  s(X) :- t(X,X).
10. t(a,b).
11. t(b,a).
```

Requête :

```
?-p(X,X).
```

Preuve :

```
?-q(X,Z),r(Z,X).    [1 - Y=X]
?-r(a,b).           [3 - X=b, Z=a]
fail.
?-r(b,X).           [4 - Z=b]
?- success.         [6. X=a]
```

SLD-résolution (cas général)

Arbre de résolution Pour explorer l'ensemble des preuves possibles, on construit un arbre :

- La racine est la top-clause $\neg Q$ (qu'on note R_0), initialement ouverte
- On itére jusque saturation
  - Pour chaque feuille ouverte $R_i \neq \square$, on choisit **un** littéral $L \in R_i$ et on détermine l'ensemble des clauses D_i^k de Σ ayant un littéral unifiable à $\neg L$
  - Si cet ensemble est vide, on ferme le noeud (fail).
  - Sinon, on construit pour chaque D_i^k de l'ensemble, un noeud fils $R_{i+1} = Res(R_i, D_i^k, \sigma_i^k)$

Chaque chemin $R_1^k, \ldots R_n^k$ de la racine (exclue) vers une feuille $\square$ donne une preuve par SLD-résolution de $\sigma(Q)$ (où σ est la composition de tous les σ_i^k sur le chemin).
Si toutes les feuilles sont fermées, il n'y a pas de preuves de Q à partir de Σ (renvoie fail).
Si l'on cherche une seule preuve, on peut s'arrêter dès que l'on trouve $\square$.

Exemple

```
1. p(X,Y) :- q(X,Z),r(Z,Y).
2. p(X,X) :- s(X).
3. q(b,a).
4. q(X,b).
5. q(X,a) :- r(a,X).
6. r(b,a).
```

```
7.  s(X) :- t(X,a).
8.  s(X) :- t(X,b).
9.  s(X) :- t(X,X).
10. t(a,b).
11. t(b,a).
```

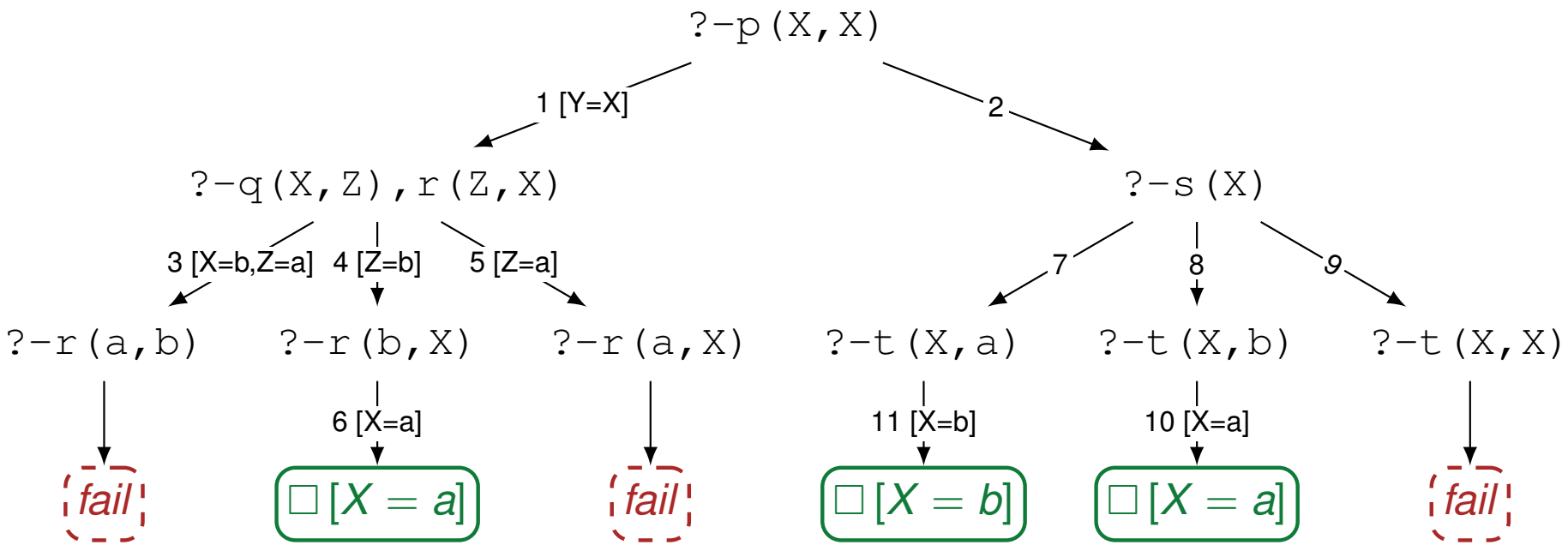
Requête : `?-p(X,X).`

Exemple

```
1. p(X,Y) :- q(X,Z),r(Z,Y).
2. p(X,X) :- s(X).
3. q(b,a).
4. q(X,b).
5. q(X,a) :- r(a,X).
6. r(b,a).
```

```
7.  s(X) :- t(X,a).
8.  s(X) :- t(X,b).
9.  s(X) :- t(X,X).
10. t(a,b).
11. t(b,a).
```

Requête : `?-p(X,X).`

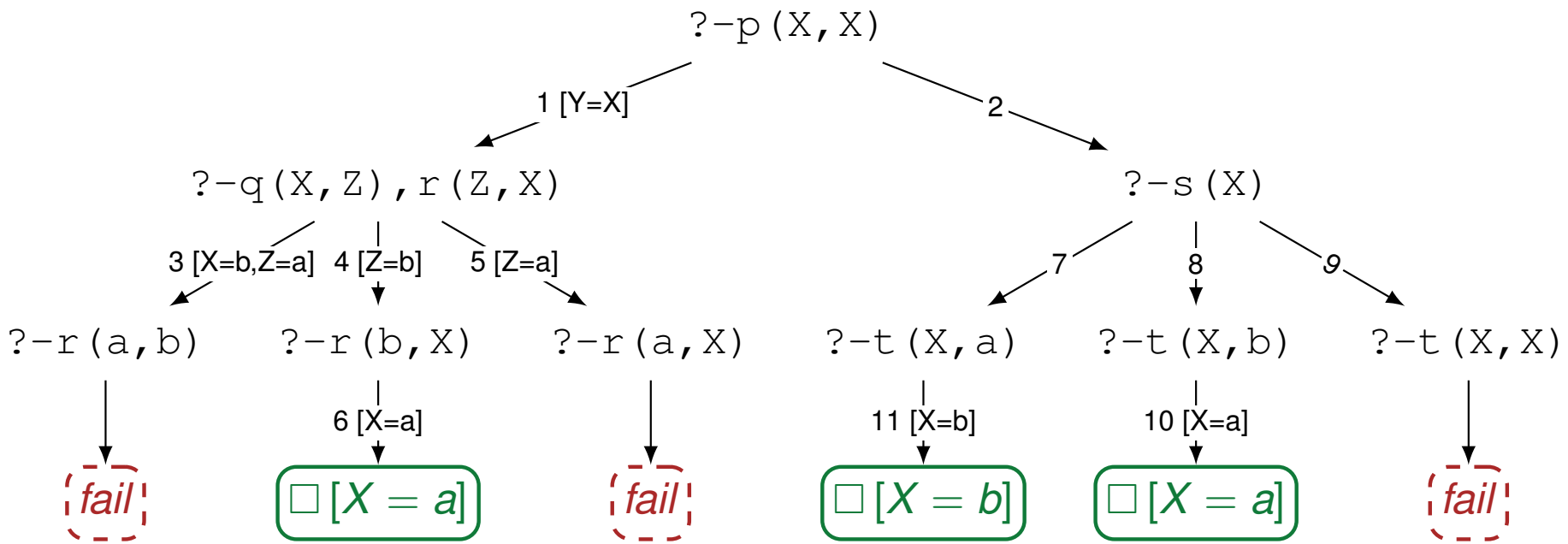


Exemple

```
1. p(X,Y) :- q(X,Z),r(Z,Y).
2. p(X,X) :- s(X).
3. q(b,a).
4. q(X,b).
5. q(X,a) :- r(a,X).
6. r(b,a).
```

```
7.  s(X) :- t(X,a).
8.  s(X) :- t(X,b).
9.  s(X) :- t(X,X).
10. t(a,b).
11. t(b,a).
```

Requête : `?-p(X,X).`



Réponse : `X=a ; X=b ; X=a ; false.`